

Strukturalni paterni

1. Proxy Patern

- **Opis:** Služi kao zamjenski objekt koji kontroliše pristup originalnom objektu. U našem slučaju, **TransakcijaProxy** kontroliše pristup klasi **Transakcija**.
- **Primjena u našem sistemu:** Koristi se za zaštitu finansijskih operacija. Proxy provjerava da li korisnik ima dovoljno sredstava na računu (atribut *stanjeRacuna*) i da li ima pravo da izvrši uplatu prije nego što se stvarni proces plaćanja pokrene.

2. Decorator Patern

- **Opis:** Omogućava dinamičko dodavanje novih funkcionalnosti objektu bez mijenjanja njegove strukture ili korištenja nasljeđivanja.
- **Primjena:** Primjenjuje se na klasu **Obavjestenje**. Obavijestima se mogu dodavati neki posebni ukrasi. Umjesto da mijenjamo klasu obavijest, može se koristiti nekoliko dekoratora koji dodaju nove funkcionalnosti.

3. Adapter Patern

- **Opis:** Omogućava saradnju između interfejsa koji su inače nekompatibilni. Radi kao "pretvarač".
- **Primjena:** Ako sistem u budućnosti bude koristio eksterni servis za generisanje QR kodova (metoda *generisiQRKod* u klasi Termin) koji ima drugačiji format podataka, Adapter bi omogućio da naš sistem komunicira sa tim servisom bez mijenjanja logike u klasi Termin.

4. Facade Patern

- **Opis:** Pruža jednostavan interfejs kompleksnom podsistemu (skupu klasa).
- **Primjena:** Može se koristiti za proces rezervacije termina. Umjesto da klijentski kod mora direktno komunicirati sa klasama Korisnik, Termin i Transakcija istovremeno, **RezervacijaFacade** bi pružila jednu metodu *rezervisiSve()* koja interno upravlja svim tim objektima.

5. Bridge Patern

- **Opis:** Odvaja apstrakciju od njene implementacije kako bi obje strane mogle nezavisno da se mijenjaju.
- **Primjena:** U našem sistemu, Bridge bi se mogao koristiti za sistem plaćanja. Razdvojili bi apstrakciju (proces uplate) od same implementacije načina plaćanja (kartica, gotovina). To bi omogućilo dodavanje novih metoda plaćanja bez mijenjanja osnovne logike transakcije.

6. Composite Patern

- **Opis:** Omogućava da se objekti organizuju u hijerarhiju (stablo), gdje se pojedinačni i složeni objekti tretiraju na isti način.
- **Primjena:** Služio bi nam da možemo npr. predmete ili materijale grupisati u veće module ili kategorije. Na taj način možemo izvršavati istu operaciju (npr. izračunaj sate ili cijenu) nad jednim predmetom ili nad čitavom grupom predmeta odjednom.